



Aula 4 (Parte 2) - Medindo diretamente o tempo da primitiva

🕒 Criado em	7 de setembro de 2026 11:39
🏷 Tags	

Antes, o professor fazia algo assim:

```
medir o tempo do laço inteiro
↓
repetir 100 vezes
↓
tratar os tempos
↓
obter uma média
```

Agora ele está tentando fazer algo diferente:

Medir diretamente o tempo da atribuição que queremos estudar.

Isso é uma mudança bem importante.

1. Primeiro, vamos olhar para as duas linhas comentadas

A colega recebeu:

```
#FOCAlaço = (0, n+1, n+1, n + 5)
# FOCA = (0, n+1, n+1, n+5 +n)
```

Isso não é código sendo executado. São anotações para lembrar a contagem.

Na aula anterior chegamos a:

```
FOCAlaço = (0, n+1, n+1, n+5)
```

Lembrando a ordem:

```
(F, 0, C, A)
```

Então:

```
F = 0
0 = n + 1
C = n + 1
A = n + 5
```

Isso representa **o gerenciamento do laço**.

2. Por que aparece **n + 5** em A?

Essa é uma daquelas coisas que não vale decorar sem entender.

Para:

```
for i in range(n):
```

existe um conjunto de operações necessárias para controlar o laço.

Temos coisas como:

```
atribuição do limite
atribuição do início
atribuição do incremento
inicialização de i
comparações
```

```
incrementos
...
```

Por isso a quantidade de atribuições não é simplesmente n .

O professor chegou, pela forma de contagem que ele está usando, em:

```
A = n + 5
```

Então:

```
FOCALaço = (0, n+1, n+1, n+5)
```

é o **custo do gerenciamento do laço**.

3. E então aparece:

```
# FOCA = (0, n+1, n+1, n+5 +n)
```

Por quê?

Porque agora temos uma atribuição adicional dentro do laço:

```
for i in range(n):
    x = 1
```

O gerenciamento do laço já tinha:

```
A = n + 5
```

Mas:

```
x = 1
```

é executado n vezes.

Portanto temos:

```
atribuições do laço =  $n + 5$   
atribuições de  $x = n$ 
```

Somando:

```
 $A = n + 5 + n$ 
```

OU:

```
 $A = 2n + 5$ 
```

Então:

```
FOCA = (0,  $n+1$ ,  $n+1$ ,  $2n+5$ )
```

Essa é a contagem de primitivas do **laço + corpo**.

4. Só que agora o professor quer algo diferente

Veja:

```
T0U_A = []
```

Esse nome provavelmente está sendo usado para guardar os tempos obtidos para a atribuição.

Podemos interpretar mentalmente:

```
T0U_A  
  ↓  
tempos da operação A
```

E novamente:

```
R = 100
n = 100000
```

Temos:

```
R = quantidade de experimentos
n = quantidade de execuções da atribuição em cada experimento
```

5. O primeiro **for**

```
for r in range(R):
```

Como:

```
R = 100
```

teremos até 100 experimentos.

Cada experimento vai tentar descobrir:

| Quanto tempo, em média, uma atribuição leva?

6. Agora aparece uma coisa muito importante

Dentro dele:

```
t = 0
```

Depois:

```
for i in range(n):
```

E aí:

```
tic = time()
x = 1
tac = time()
t += tac - tic
```

Observe a diferença em relação ao código anterior.

Antes tínhamos:

```
tic = perf_counter()

for i in range(n):
    x = 1

toc = perf_counter()
```

Ou seja:

```
MEDIR
↓
todo o laço
↓
PARAR
```

Agora temos:

```
for i in range(n):
    tic = time()
    x = 1
    tac = time()
```

Ou seja:

```
para cada execução:

começa cronômetro
↓
x = 1
↓
termina cronômetro
```

Isso é uma mudança enorme na estratégia.

7. O que o professor está tentando fazer?

Ele quer medir **somente o trecho**:

```
x = 1
```

O cronômetro começa imediatamente antes:

```
tic = time()
```

e termina imediatamente depois:

```
tac = time()
```

Então:

```
tempo da atribuição ≈ tac - tic
```

Esse valor é acumulado:

```
t += tac - tic
```

8. Por que acumular?

Imagine que uma execução produza:

```
0.000001
```

segundos.

Esse valor é extremamente pequeno.

Então, em vez de confiar em uma única medição, fazemos:

```
medição 1
+
medição 2
+
medição 3
+
...
+
medição 100000
```

Isso acontece através de:

```
t += tac - tic
```

Ou seja:

```
t = t + (tempo da atribuição)
```

No final do `for`:

```
t
```

representa aproximadamente:

| a soma dos tempos medidos das `n` execuções de `x = 1`.

9. E então aparece uma linha fundamental

```
T0U_A.append(t/n)
```

Aqui o professor divide o tempo total por `n`.

Por quê?

Porque tivemos `n` medições da atribuição.

Então:

```
tempo total das n atribuições
-----
```


n

resulta em:

tempo médio por atribuição

Ou seja:

t / n

é uma estimativa experimental do tempo médio da primitiva A.

Esse valor vai para:

T0U_A

10. Agora conseguimos enxergar a intenção de **T0U_A**

Depois de vários experimentos, teremos algo conceitualmente assim:

```
T0U_A = [  
    tempo médio do experimento 1,  
    tempo médio do experimento 2,  
    tempo médio do experimento 3,  
    ...  
]
```

Por exemplo, imaginariamente:

```
T0U_A = [  
    0.000000020,  
    0.000000019,  
    0.000000021,  
    0.000000020,  
    ...  
]
```

Cada elemento representa:

| uma estimativa do tempo médio de uma atribuição naquele experimento.

11. Agora voltamos para a estatística

Depois de cada experimento:

```
media = np.mean(TOU_A)
sigma = np.std(TOU_A)
cv = sigma/media
```

Aqui o professor está olhando para a **distribuição dos valores de** `TOU_A`.

Temos:

```
TOU_A
↓
vários tempos médios
↓
média
↓
desvio padrão
↓
coeficiente de variação
```

12. Por que fazer isso de novo?

Porque mesmo depois de dividir por `n`, ainda podemos ter variações.

Por exemplo:

```
Experimento 1 → 20 ns
Experimento 2 → 21 ns
Experimento 3 → 19 ns
Experimento 4 → 20 ns
Experimento 5 → 50 ns
```

O `50` pode ser uma medição problemática.

Então o professor quer verificar:

| "As estimativas que estou obtendo são estáveis?"

13. O critério continua sendo o coeficiente de variação

Temos:

```
if cv < 0.15:  
    break
```

Ou seja:

```
se CV < 15%  
    ↓  
    consideramos a medição suficientemente estável  
    ↓  
    paramos o experimento
```

Então existe uma diferença interessante em relação ao código anterior.

Antes o professor estava filtrando **os tempos individuais**.

Agora ele está acumulando **estimativas médias de cada experimento** e verificando a estabilidade delas.

14. Essa diferença é MUITO importante

Vamos comparar.

Estratégia anterior

Imagine:

```
100 tempos individuais
```

O professor fazia:

```
tempos  
    ↓  
    média  
    ↓  
    desvio  
    ↓  
    corte
```

↓
filtragem

Estratégia atual

Agora temos:

```
Experimento 1 → média das n atribuições  
Experimento 2 → média das n atribuições  
Experimento 3 → média das n atribuições  
...
```

Então:

```
cada experimento  
  ↓  
mede n atribuições  
  ↓  
soma os tempos  
  ↓  
divide por n  
  ↓  
obtem uma média  
  ↓  
guarda em TOU_A
```

Depois:

```
TOU_A  
  ↓  
média das médias  
  ↓  
desvio  
  ↓  
CV
```

Isso é uma forma diferente de estabilizar a medição.

15. Mas existe uma questão MUITO interessante nesse código

Olha novamente:

```
tic = time()
x = 1
tac = time()
```

O professor está tentando medir `x = 1`.

Mas o próprio ato de medir:

```
time()
```

também possui custo.

Então a medição não é perfeitamente:

```
tempo de x = 1
```

Ela é mais próxima de:

```
tempo de chamar time()
+
tempo de x = 1
+
tempo de chamar time()
```

Ou seja:

```
tempo medido
=
custo da medição
+
custo da atribuição
+
custo da medição
```

Isso é um problema clássico de **medição experimental de operações muito pequenas**.

E isso ajuda a entender por que o professor está usando `n = 100000`.

Ele está tentando obter uma medida mais útil repetindo a operação muitas vezes.

16. Existe outro detalhe: `time()` versus `perf_counter()`

Na atividade anterior tínhamos:

```
from time import perf_counter
```

Agora:

```
from time import time
```

São funções diferentes para obter uma referência temporal.

Para medições de pequenos intervalos, `perf_counter()` costuma ser a ferramenta mais apropriada para benchmarking porque foi projetada para medir intervalos de tempo com alta resolução.

Então, se você encontrar essa diferença nas anotações, **não significa que o professor mudou o conceito de análise.**

A ideia continua sendo:

```
tic → início  
tac → fim  
tac - tic → duração
```

17. Agora vamos ligar isso ao FOCAlaço

Aqui está a parte que provavelmente o professor queria começar a conectar.

Nós sabemos:

```
FOCAlaço = (0, n+1, n+1, n+5)
```

E:

```
x = 1
```

acontece `n` vezes.

Então:

```
FOCA = (0, n+1, n+1, 2n+5)
```

Agora imagine que conseguimos descobrir experimentalmente:

```
T0U_A
```

ou seja:

| tempo médio de uma atribuição.

Então podemos começar a construir uma função de tempo.

18. Exemplo conceitual

Suponha que, experimentalmente, descobrimos:

```
tempo médio da atribuição = 0,00000002 s
```

E nosso programa possui:

```
2n + 5 atribuições
```

Então a contribuição das atribuições seria:

```
(2n + 5) × T0U_A
```

Mas ainda temos:

```
n + 1 operações  
n + 1 comparações
```

Então, se também tivermos:

T0U_0
T0U_C

podemos construir:

```
T(n) =  
(n + 1) × T0U_0  
+  
(n + 1) × T0U_C  
+  
(2n + 5) × T0U_A
```

E pronto.

Estamos voltando exatamente para o objetivo que vinha sendo construído desde as primeiras aulas.

19. Então qual é o papel desse experimento?

Podemos resumir assim:

```
      CÓDIGO  
      ↓  
contar primitivas  
      ↓  
FOCA(n)  
      ↓  
precisamos dos τ  
      ↓  
fazer experimento  
      ↓  
medir tempo das primitivas  
      ↓  
obter T0U_A  
      ↓  
usar T0U_A na função T(n)
```

Essa é a grande sacada.

20. Por que o professor faz **n = 100000** ?

Agora isso fica muito mais intuitivo.

Ele quer medir uma operação extremamente rápida:

```
x = 1
```

Se tentar medir apenas uma vez:

```
tic
↓
x = 1
↓
tac
```

o intervalo pode ser tão pequeno que fica difícil obter uma medida confiável.

Então ele faz:

```
100000 vezes
```

e calcula:

```
tempo total
-----
100000
```

Isso produz uma estimativa do tempo médio por operação.

21. É quase como pesar vários grãos juntos

Uma analogia boa:

Imagine que você queira descobrir quanto pesa **um único grão de arroz**, mas sua balança não consegue medir um grão com precisão.

Você faz:

```
100.000 grãos
↓
pesa todos
```

```
↓  
divide por 100.000  
↓  
peso médio de um grão
```

O professor está fazendo algo conceitualmente parecido:

```
100.000 atribuições  
↓  
mede o tempo total  
↓  
divide por 100.000  
↓  
tempo médio de uma atribuição
```

Isso é exatamente o motivo de usar um `n` grande.

22. E por que repetir 100 vezes?

Porque mesmo essa média pode variar.

Então:

```
100.000 atribuições  
↓  
média do experimento 1  
  
100.000 atribuições  
↓  
média do experimento 2  
  
...  
  
100.000 atribuições  
↓  
média do experimento 100
```

Depois:

```
TOU_A  
↓  
média  
↓  
desvio
```

↓
CV

Até que:

CV < 15%

Ou seja, o professor está procurando uma medição suficientemente estável.

23. A diferença entre `t` e `TOU_A`

Isso pode confundir bastante, então vamos separar.

`t`

`t = 0`

É o acumulador do **tempo das n execuções dentro de um experimento**.

Depois:

`t += tac - tic`

vai acumulando.

No final:

`t/n`

é o tempo médio de uma atribuição **naquele experimento**.

`TOU_A`

`TOU_A = []`

Guarda os resultados de vários experimentos.

Então:

```
t
↓
tempo total de um experimento

t/n
↓
tempo médio daquele experimento

TOU_A
↓
tempo médio de vários experimentos
```

Essa distinção é fundamental.

24. O fluxo completo desse código

Agora podemos enxergar o programa inteiro:

```
R = 100
n = 100000

    ↓

COMEÇA UM EXPERIMENTO

    ↓

t = 0

    ↓

REPETE n VEZES

    ↓

tic
↓
x = 1
↓
tac
↓
t += tac - tic

    ↓

terminou n vezes

    ↓

t / n
```

↓
tempo médio da atribuição
nesse experimento

↓
adiciona em T0U_A

↓
calcula média
calcula desvio
calcula CV

↓
CV < 15%?

NÃO	SIM
↓	↓
continua	para

25. O que eu acho que você deve guardar dessa parte

Se cair um código parecido na prova, pense nesta sequência:

1. O que está sendo medido?

Aqui:

x = 1

Uma atribuição.

2. Quantas vezes?

n = 100000

3. Por que tantas vezes?

Para tornar a medição de uma operação muito rápida mais representativa.

4. O que é **t** ?

Soma dos tempos medidos naquele experimento.

5. O que é **t/n** ?

Tempo médio de uma atribuição naquele experimento.

6. O que é **TOU_A** ?

Lista contendo as médias obtidas em vários experimentos.

7. O que são **media** , **sigma** e **cv** ?

Medidas estatísticas usadas para verificar a estabilidade dos resultados.

8. Por que **cv < 0.15** ?

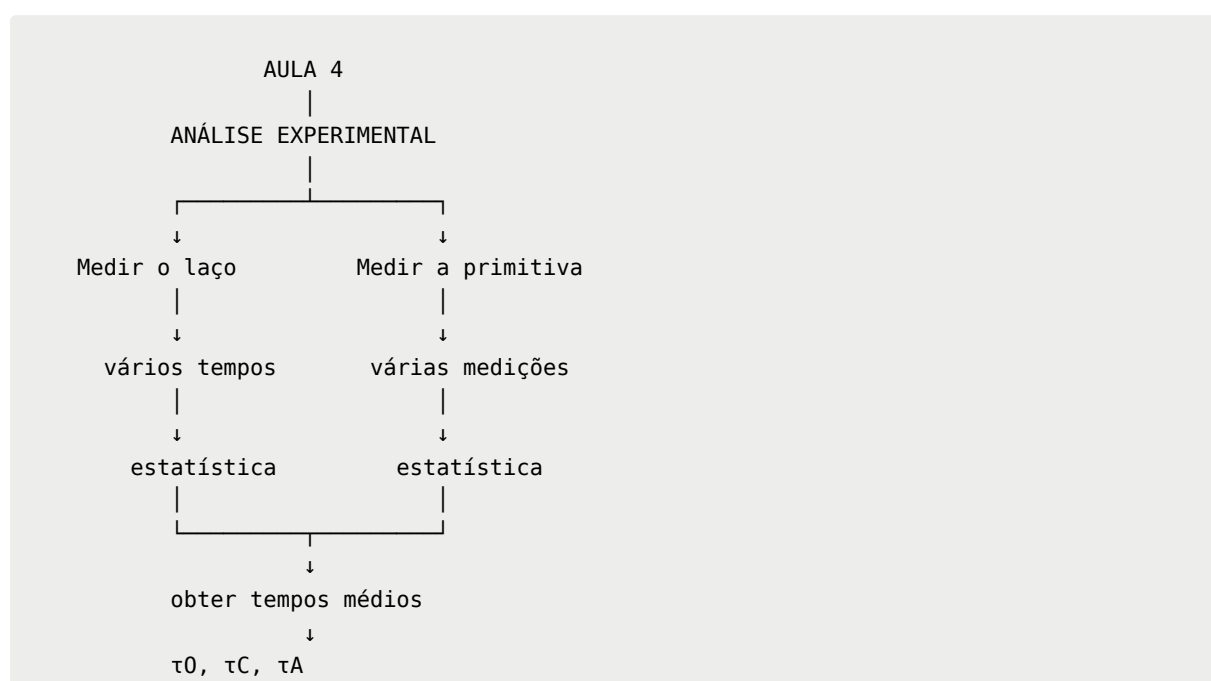
Para aceitar os resultados quando a variação relativa fica abaixo de 15%.

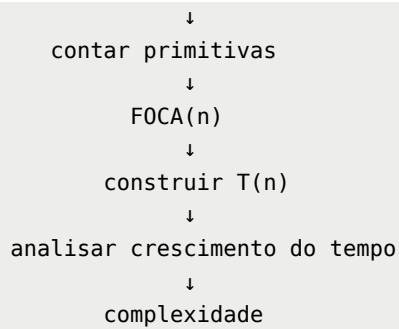
9. E onde entra FOCA?

A contagem diz **quantas primitivas existem**.

O experimento tenta descobrir **quanto tempo cada tipo de primitiva leva**.

26. E aqui está o mapa que eu usaria para estudar a Aula 4 inteira





Então, sim: **essa segunda parte é a continuação natural da primeira**, mas agora o professor está dando um passo mais profundo: ele não quer apenas medir "quanto tempo o laço leva"; ele está tentando **isolar experimentalmente o custo de uma primitiva** para depois poder usar esse custo na estimativa do tempo de programas maiores.

E aquela anotação `FOCAlaço` → `FOCA` é justamente a ponte entre "**quanto custa cada primitiva**" e "**quanto custa o programa inteiro**".